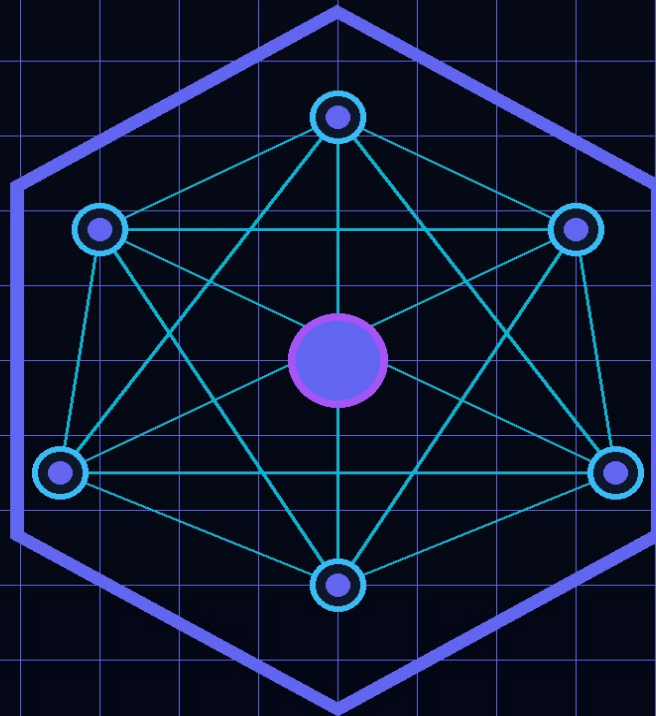


ADVANCED DISTRIBUTED SYSTEMS & ARCHITECTURE

HEXAGEN ECOSYSTEM

MICROSERVICES



Event-Driven Architecture, CQRS at Scale
& Eventual Consistency with Distributed Slices

RABBITMQ • APACHE KAFKA • GRPC • SAGAS • ZERO-TRUST

BY / POR

Raul Alejandro Baez Camarillo

HexaGen: Microservices and Distributed Slices

Event-Driven Architecture, CQRS at Scale, and Eventual Consistency

By Raul Alejandro Baez Camarillo

Book Synopsis

Migrating from a monolith to a distributed architecture is one of the greatest challenges in contemporary software engineering. Most teams fail by decomposing prematurely or creating "distributed monoliths": systems tightly coupled across the network with high latency, cascading failures, and data inconsistencies.

HexaGen: Microservices and Distributed Slices delivers an authoritative, production-grade roadmap to decompose **Vertical Slice Architecture (VSA)** monoliths into autonomous, event-driven microservices engineered to handle millions of transactions per second with strict domain boundaries and zero operational friction.

Target Audience

- **Software Architects & Tech Leads:** Who need to design resilient, scalable, and fully auditable distributed systems.
 - **Senior Backend Engineers:** Seeking mastery over Event-Driven Architecture (EDA), CQRS at Scale, Transactional Outbox, Sagas, and Event Sourcing.
 - **HexaGen Ecosystem Developers:** Wanting to scale their Vertical Slices into high-throughput distributed microservices without sacrificing simplicity.
-

Tech Stack & Key Architectural Patterns

- **Languages & Runtimes:** PHP 8.3+, FrankenPHP, Go (High-concurrency streaming workers).
 - **Messaging & Event Streaming:** RabbitMQ (AMQP 0-9-1), Apache Kafka, Redis Streams.
 - **Protocols & Transport:** gRPC / Protocol Buffers, HTTP/3, Server-Sent Events (SSE).
 - **Core Patterns:** Distributed Vertical Slices, CQRS, Transactional Outbox, Orchestrated & Choreographed Sagas, Event Sourcing, Circuit Breakers, Zero-Trust mTLS.
-

© 2026 Raul Alejandro Baez Camarillo • VortexSolutions • Hexagen Ecosystem. All rights reserved.

Chapter 1: From Monolithic Vertical Slices to Distributed Slices

The natural evolution of a high-throughput web application built on **Vertical Slice Architecture (VSA)** eventually reaches a critical juncture: when is the right time to extract slices into autonomous microservices, and how can we execute this without falling into the "distributed monolith" trap?

1. The Three Drivers of Decomposition

Decomposing a system must never follow tech fashion; it must answer quantifiable business and operational demands:

- **Asymmetric Scalability:** A specific slice (such as payment webhook ingestion or real-time IoT telemetry) requires 50x more CPU and memory resources than the rest of the application.
 - **Deployment Velocity & Team Autonomy (Conway's Law):** Independent product teams need to deploy multiple times per day without release locks or coordinating monolithic deployments.
 - **Critical Fault Isolation:** Memory leaks or fatal errors in a non-transactional module (e.g. heavy PDF report generation) must never bring down core checkout or payment gateways.
-

2. Architectural Transition Map

Architectural Tier	Monolithic Vertical Slice	Distributed Microservices
Code Organization	Self-contained slice directories (<code>app/Slices/...</code>)	Autonomous bounded context repositories / services
Persistence	Shared database with isolated tables per slice	Dedicated database per service (<i>Database-per-Service</i>)
Communication	In-memory handler execution	Asynchronous integration events & gRPC contracts
Transactional Boundaries	Local ACID transactions	Eventual Consistency & the Saga Pattern with compensation

3. Bounded Context Boundaries

In HexaGenPHP, each Vertical Slice already possesses high cohesion and low coupling. When transitioning to distributed slices, we group related slices inside a strict **Domain-Driven Design (DDD) Bounded Context**.

Golden Rules of Distributed Slices:

1. **Database-per-Service:** No microservice ever reads or writes directly to another service's database schema. All communication happens via APIs or event streams.
2. **No Shared Domain Entities:** Integration DTOs are immutable public contracts. Never share Doctrine or Eloquent ORM entities across repositories.
3. **Temporal Decoupling:** A service must continue accepting and processing requests even when downstream dependencies are temporarily unavailable.

Chapter 2: Anatomy of a HexaGen Microservice

A microservice within the HexaGen ecosystem preserves the clean philosophy of ports and adapters combined with Vertical Slice Architecture, augmented with distributed transport abstractions.

1. Directory Structure of a Distributed Service

```
order-service/  
├── src/  
│   ├── Domain/                                # Pure Business Rules  
│   │   ├── Model/Order.php  
│   │   ├── ValueObjects/Money.php  
│   │   └── Ports/  
│   │       ├── OrderRepositoryPort.php  
│   │       └── MessagePublisherPort.php  
│   ├── Slices/                                # Use Cases (VSA)  
│   │   ├── CreateOrder/  
│   │   │   ├── CreateOrderCommand.php  
│   │   │   ├── CreateOrderHandler.php  
│   │   │   └── CreateOrderEndpoint.php  
│   │   └── CancelOrder/  
│   ├── Infrastructure/                        # Technology Adapters  
│   │   ├── Persistence/DoctrineOrderRepository.php  
│   │   ├── Messaging/RabbitMQMessagePublisher.php  
│   │   └── Grpc/OrderGrpcServer.php  
│   └── Shared/  
│       └── Contracts/Events/OrderCreatedIntegrationEvent.php
```

2. Implementing Integration Ports

```
namespace OrderService\Domain\Ports;

use OrderService\Shared\Contracts\Events\IntegrationEventInterface;

interface MessagePublisherPort
{
    public function publish(IntegrationEventInterface $event): void;
    public function publishBatch(array $events): void;
}
```

The core domain generates pure business events, which are converted into serializable integration contracts before being published across the wire:

```
namespace OrderService\Slices\CreateOrder;

use OrderService\Domain\Model\Order;
use OrderService\Domain\Ports\OrderRepositoryPort;
use OrderService\Domain\Ports\MessagePublisherPort;
use OrderService\Shared\Contracts\Events\OrderCreatedIntegrationEvent;

final readonly class CreateOrderHandler
{
    public function __construct(
        private OrderRepositoryPort $repository,
        private MessagePublisherPort $publisher
    ) {}

    public function handle(CreateOrderCommand $cmd): string
    {
        $order = Order::create($cmd->customerId, $cmd->items);
        $this->repository->save($order);

        // Publish integration event to broker
        $this->publisher->publish(new OrderCreatedIntegrationEvent(
            orderId: $order->id(),
            customerId: $order->customerId(),
            totalAmount: $order->total()->amount(),
            currency: $order->total()->currency(),
            occurredOn: new \DateTimeImmutable()
        ));

        return $order->id();
    }
}
```

Chapter 4: CQRS at Scale & Decoupled Read Models

Command Query Responsibility Segregation (CQRS) empowers architects to optimize write models strictly for transactional invariants and read models for ultra-fast, denormalized retrieval.

1. The Distributed Query Problem

In a monolith, fetching an aggregated report with customer, order, and payment details requires a single SQL `JOIN`. In microservices, executing multiple synchronous HTTP calls across network boundaries creates:

- **Multiplicative latency:** If 4 services take 50 ms each, the query accumulates over 200 ms.
 - **Cascading fragility:** If any downstream dependency is down, the entire user query fails.
-

2. Asynchronous Read Projection Workflow

1. **Write Side (Command):** The Order Service validates and executes the business command, committing changes to its transactional database (e.g. PostgreSQL).
 2. **Event Emission:** The service publishes an `OrderCreated` integration event to the message broker.
 3. **Projection Worker:** A dedicated background worker consumes the event and updates the denormalized read model in an optimized store (e.g. Elasticsearch or Redis).
 4. **Read Side (Query):** The Query API serves the client request in under 5 milliseconds directly from the projection without touching the transactional write database.
-

3. Implementing a Materialized Read Projection

```
namespace ReportingService\Projections;

use ReportingService\Infrastructure\ElasticsearchClient;
use OrderService\Shared\Contracts\Events\OrderCreatedIntegrationEvent;

final readonly class OrderSummaryProjector
{
    public function __construct(private ElasticsearchClient $elastic) {}

    public function handle(OrderCreatedIntegrationEvent $event): void
    {
        $document = [
            'order_id' => $event->orderId,
            'customer_id' => $event->customerId,
            'total_amount' => $event->totalAmount,
            'status' => 'PENDING_PAYMENT',
            'created_at' => $event->occurredOn-
            >format(\DateTimeInterface::ATOM)
        ];

        $this->elastic->index('orders_view', $event->orderId, $document);
    }
}
```


Chapter 5: Eventual Consistency & The Transactional Outbox Pattern

The greatest hazard in distributed architectures is the dual-write problem: what happens if the database saves the order record, but the server crashes before dispatching the event to RabbitMQ? Or what if the message sends, but the SQL transaction rolls back?

1. The Transactional Outbox Pattern

To achieve 100% data consistency without distributed two-phase commits (2PC), events are written to a dedicated `outbox_events` table inside the exact same local SQL transaction as the domain entity.

Two-Phase Execution Flow:

- **Phase 1 (Atomic Local Transaction):**
 - `INSERT INTO orders (...) VALUES (...);`
 - `INSERT INTO outbox_events (event_type, payload, status) VALUES (...);`
 - `COMMIT;` (Both succeed or both fail together).
 - **Phase 2 (Asynchronous Relay):**
 - An independent background relay process polls pending outbox records, dispatches them to the broker, and marks them as `PROCESSED`.
-

2. SQL Schema for Transactional Outbox

```
CREATE TABLE outbox_events (  
  id CHAR(36) PRIMARY KEY,  
  event_type VARCHAR(255) NOT NULL,  
  aggregate_id VARCHAR(64) NOT NULL,  
  payload JSON NOT NULL,  
  status ENUM('PENDING', 'PROCESSED', 'FAILED') DEFAULT 'PENDING',  
  retry_count INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  processed_at TIMESTAMP NULL,  
  INDEX idx_status_created (status, created_at)  
);
```

3. High-Throughput Outbox Dispatcher in PHP 8.3

```
namespace HexaGen\Distributed\Outbox;

use PDO;

final readonly class OutboxRelay
{
    public function __construct(
        private PDO $db,
        private MessagePublisherInterface $publisher
    ) {}

    public function processBatch(int $limit = 100): int
    {
        $stmt = $this->db->prepare("
            SELECT id, event_type, payload
            FROM outbox_events
            WHERE status = 'PENDING'
            ORDER BY created_at ASC
            LIMIT :limit FOR UPDATE SKIP LOCKED
        ");
        $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
        $stmt->execute();
        $events = $stmt->fetchAll(PDO::FETCH_ASSOC);

        foreach ($events as $event) {
            $this->publisher->publish($event['event_type'],
                $event['payload']);

            $update = $this->db->prepare("
                UPDATE outbox_events
                SET status = 'PROCESSED', processed_at = NOW()
                WHERE id = :id
            ");
            $update->execute([':id' => $event['id']]);
        }

        return count($events);
    }
}
```

Chapter 6: Distributed Transactions: The Saga Pattern

When a business process spans across multiple microservices (e.g. Order Checkout → Charge Credit Card → Reserve Warehouse Inventory → Dispatch Email Invoice), traditional ACID transactions are impossible. The **Saga Pattern** coordinates a sequence of local transactions paired with **compensating transactions**.

1. Choreography vs. Orchestration

Dimension	Choreographed Saga	Orchestrated Saga
Control Model	Decentralized; services react to partner events	Centralized; a dedicated Coordinator drives steps
Infrastructure Coupling	Minimal direct service-to-service links	Slightly higher coupling to the central state machine
Traceability & Visibility	Difficult to monitor beyond 4-5 steps	Highly explicit, auditable state machine
Recommended For	Simple workflows (2-3 services)	Complex, multi-step financial and logistics sagas

2. Orchestrated Saga State Machine in PHP 8.3

```
namespace OrderService\Sagas\Checkout;

final class CheckoutSagaOrchestrator
{
    public function __construct(
        private PaymentClientPort $payments,
        private InventoryClientPort $inventory,
        private NotificationClientPort $notifications
    ) {}

    public function execute(CheckoutSagaData $data): void
    {
        // Step 1: Authorize payment
        $paymentResult = $this->payments->authorizePayment($data->orderId,
        $data->amount);
        if (!$paymentResult->isSuccess()) {
            $this->compensate($data, 'PAYMENT_FAILED');
            return;
        }

        // Step 2: Reserve inventory in warehouse
        $stockResult = $this->inventory->reserveItems($data->orderId,
        $data->items);
        if (!$stockResult->isSuccess()) {
            // COMPENSATING ACTION: Refund charged payment
            $this->payments->refundPayment($data->orderId, $data->amount);
            $this->compensate($data, 'INSUFFICIENT_STOCK');
            return;
        }

        // Step 3: Dispatch confirmation
        $this->notifications->sendConfirmation($data->customerId, $data->orderId);
    }

    private function compensate(CheckoutSagaData $data, string $reason):
    void
    {
        // Mark order as canceled and write forensic audit logs
    }
}
```

Chapter 7: Synchronous vs Asynchronous Communication: gRPC & REST

While asynchronous events are the cornerstone of distributed scalability, specific workflows demand synchronous, single-digit millisecond latency (e.g. real-time identity verification, token introspection, or risk scoring).

1. Why gRPC with Protocol Buffers

- **Binary Wire Format:** Protobuf cuts serialization overhead and shrinks payload size by up to 70% compared to JSON.
- **HTTP/2 Multiplexing:** Multiple concurrent requests stream over a single underlying TCP connection.
- **Strict Interface Contracts:** `.proto` schema definitions serve as the authoritative single source of truth across PHP, Go, Rust, and Node.js.

```
syntax = "proto3";

package hexagen.payments.v1;

service PaymentService {
  rpc ProcessPayment (PaymentRequest) returns (PaymentResponse);
}

message PaymentRequest {
  string transaction_id = 1;
  string customer_id = 2;
  double amount = 3;
  string currency = 4;
}

message PaymentResponse {
  string status = 1;
  string authorization_code = 2;
  int64 processed_at_timestamp = 3;
}
```

2. Consuming gRPC Services in PHP 8.3

```
namespace App\Infrastructure\Adapters;

use Hexagen\Payments\V1\PaymentServiceClient;
use Hexagen\Payments\V1\PaymentRequest;
use App\Domain\Ports\PaymentGatewayPort;

final readonly class GrpcPaymentGatewayAdapter implements
PaymentGatewayPort
{
    private PaymentServiceClient $client;

    public function __construct(string $hostname)
    {
        $this->client = new PaymentServiceClient($hostname, [
            'credentials' => \Grpc\ChannelCredentials::createInsecure(),
        ]);
    }

    public function charge(string $txId, string $customer, float $amount,
string $currency): bool
    {
        $request = new PaymentRequest();
        $request->setTransactionId($txId);
        $request->setCustomerId($customer);
        $request->setAmount($amount);
        $request->setCurrency($currency);

        [$response, $status] = $this->client->ProcessPayment($request)-
>wait();

        if ($status->code !== \Grpc\STATUS_OK) {
            throw new \RuntimeException("gRPC Payment Call Failed: " .
$status->details);
        }

        return $response->getStatus() === 'APPROVED';
    }
}
```

Chapter 8: Event Sourcing: State as an Immutable Event Stream

In mission-critical financial, logistics, and audit systems, executing destructive SQL `UPDATE` operations destroys temporal business context. **Event Sourcing** models the state of an aggregate as the cumulative playback of an append-only stream of immutable events.

1. Traditional CRUD vs. Event Sourcing

Traditional CRUD

- The database holds only the latest state snapshot (`Balance: $750.00`).
- Historical context of how or why that value changed is lost without fragile audit logs.

Event Sourcing Model

- State is reconstituted by replaying the event stream:
 - `AccountOpened` (`InitialDeposit: $1,000.00`)
 - `MoneyWithdrawn` (`Amount: $300.00`)
 - `MoneyDeposited` (`Amount: $50.00`)
 - **Rebuilt Balance:** `$1,000 - $300 + $50 = $750.00`
 - **Core Benefits:** Mathematical auditability and native support for Time-Travel Debugging.
-

2. Reconstituting an Aggregate from History

```
namespace HexaGen\EventSourcing;

abstract class AggregateRoot
{
    private int $version = 0;
    private array $recordedEvents = [];

    public function recordThat(object $event): void
    {
        $this->apply($event);
        $this->recordedEvents[] = $event;
    }

    public function reconstituteFromHistory(array $events): void
    {
        foreach ($events as $event) {
            $this->apply($event);
            $this->version++;
        }
    }

    abstract protected function apply(object $event): void;

    public function releaseEvents(): array
    {
        $events = $this->recordedEvents;
        $this->recordedEvents = [];
        return $events;
    }
}
```

Chapter 9: Distributed Observability: OpenTelemetry & Tracing

In a distributed cluster, diagnosing an error across 6 independent services and 2 message brokers is impossible without unified observability grounded in the three pillars: **Structured Logs**, **Metrics**, and **Distributed Tracing**.

1. W3C TraceContext & Correlation Propagation

Every inbound request touching the API Gateway receives a standardized `traceparent` header:

```

  traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
                |      └── Trace ID ───────────┘   └── Span ID ─┘
  LFlags
  Version

```

This trace context is automatically injected into outbound HTTP requests and AMQP/Kafka message metadata, enabling comprehensive end-to-end trace waterfall visualization in tools like Jaeger and Grafana Tempo.

2. OpenTelemetry Instrumentation in PHP 8.3

```
namespace HexaGen\Observability;

use OpenTelemetry\API\Globals;
use OpenTelemetry\API\Trace\TracerInterface;

final class TracingMiddleware
{
    private TracerInterface $tracer;

    public function __construct()
    {
        $this->tracer = Globals::tracerProvider()-
>getTracer('hexagen.order_service');
    }

    public function execute(string $operationName, callable $action)
    {
        $span = $this->tracer->spanBuilder($operationName)->startSpan();
        $scope = $span->activate();

        try {
            $result = $action();
            $span-
>setStatus(\OpenTelemetry\API\Trace\StatusCode::STATUS_OK);
            return $result;
        } catch (\Throwable $e) {
            $span->recordException($e);
            $span-
>setStatus(\OpenTelemetry\API\Trace\StatusCode::STATUS_ERROR, $e-
>getMessage());
            throw $e;
        } finally {
            $scope->detach();
            $span->end();
        }
    }
}
```


Chapter 10: Resilience, Fault Tolerance & Circuit Breakers

In distributed architectures, failures are not anomalies—they are statistical certainties: networks drop packets, databases experience I/O spikes, and external APIs become unresponsive. Engineering for resilience is mandatory.

1. Circuit Breaker Lifecycle States

- **CLOSED (Normal):** Requests pass through directly to the downstream dependency. Consecutive failures are tracked.
 - **OPEN (Protected):** Once the failure threshold is breached (e.g. 5 consecutive errors), the circuit trips immediately. All subsequent calls invoke a fallback without network load.
 - **HALF-OPEN (Probing):** After a recovery timeout (e.g. 30 seconds), a single trial request is permitted through. If successful, the circuit resets to `CLOSED`; if it fails, it returns to `OPEN`.
-

2. Redis-Backed Circuit Breaker in PHP 8.3

```
namespace HexaGen\Resilience;

final class CircuitBreaker
{
    private const STATE_CLOSED = 'CLOSED';
    private const STATE_OPEN = 'OPEN';
    private const STATE_HALF_OPEN = 'HALF_OPEN';

    public function __construct(
        private \Redis $redis,
        private string $serviceName,
        private int $failureThreshold = 5,
        private int $recoveryTimeoutSeconds = 30
    ) {}

    public function call(callable $action, callable $fallback)
    {
        $state = $this->getState();

        if ($state === self::STATE_OPEN) {
            return $fallback(); // Graceful degradation without hammering
            failing downstream
        }

        try {
            $result = $action();
            $this->onSuccess();
            return $result;
        } catch (\Throwable $e) {
            $this->onFailure();
            return $fallback();
        }
    }

    private function getState(): string
    {
        return $this->redis->get("cb:{$this->serviceName}:state") ?:
        self::STATE_CLOSED;
    }

    private function onFailure(): void
    {
        $failures = $this->redis->incr("cb:{$this->serviceName}:failures");
        if ($failures >= $this->failureThreshold) {
            $this->redis->setex("cb:{$this->serviceName}:state", $this-
            >recoveryTimeoutSeconds, self::STATE_OPEN);
        }
    }

    private function onSuccess(): void
    {
        $this->redis->del("cb:{$this->serviceName}:failures");
        $this->redis->set("cb:{$this->serviceName}:state",
        self::STATE_CLOSED);
    }
}
```

Chapter 11: Zero-Trust Security & Service Mesh in Microservices

Classic perimeter security presumes all traffic inside the internal VPC is inherently safe. Under **Zero-Trust**, no microservice implicitly trusts another: every request is authenticated, encrypted, and authorized point-to-point.

1. Zero-Trust Pillars in HexaGen

1. **Mutual TLS (mTLS):** All microservices exchange cryptographic X.509 certificates verified by a private Certificate Authority before transmitting data.
 2. **Cryptographic Identity Tokens (Ed25519 JWTs):** Inter-service requests carry signed tokens verifying tenant context and role claims.
 3. **Strict Network Isolation:** Payment services can only be reached by authorized order services over dedicated network policies.
-

2. Inter-Service Token Verification in PHP 8.3

```
namespace HexaGen\Security;

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

final class InterServiceTokenVerifier
{
    public function __construct(private string $publicKeyEd25519) {}

    public function verify(string $jwtToken): ServiceIdentity
    {
        $decoded = JWT::decode($jwtToken, new Key($this->publicKeyEd25519,
            'EdDSA'));

        if ($decoded->iss !== 'hexagen-auth-authority') {
            throw new \SecurityException("Unauthorized token issuer");
        }

        return new ServiceIdentity(
            serviceId: $decoded->sub,
            tenantId: $decoded->tenant_id,
            allowedScopes: $decoded->scopes
        );
    }
}
```

Chapter 12: Production Project: Distributed Payments & Logistics Engine

To consolidate all architectural concepts learned throughout this book, we will assemble an end-to-end distributed production platform: **HexaLogistics Enterprise**.

1. System Topology Overview

- **API Gateway (Envoy):** Single entry point handling TLS termination, rate limiting, and path-based routing.
 - **Order Service (PHP 8.3 / VSA):** Order lifecycle management and transactional Outbox event dispatch to RabbitMQ.
 - **Payment Service (PHP 8.3 / VSA):** Charge authorization and checkout Saga orchestration.
 - **Catalog Service (PHP 8.3):** High-speed read queries backed by distributed Redis caching.
 - **Notification Service (Go):** High-concurrency worker for real-time customer email and webhook dispatch.
-

2. Production Docker Compose Configuration

```
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:3.13-management-alpine
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: hexagen
      RABBITMQ_DEFAULT_PASS: hexagen_secret

  redis:
    image: redis:7.2-alpine
    ports:
      - "6379:6379"

  order-service:
    build:
      context: ./services/order-service
    environment:
      DATABASE_URL: mysql://hexagen:secret@order-db:3306/orders
      RABBITMQ_HOST: rabbitmq
    depends_on:
      - rabbitmq
      - redis

  payment-service:
    build:
      context: ./services/payment-service
    environment:
      DATABASE_URL: mysql://hexagen:secret@payment-db:3306/payments
      RABBITMQ_HOST: rabbitmq
    depends_on:
      - rabbitmq
```

3. Conclusion & The Hexagen Ecosystem Horizon

You have mastered the architectural journey from monolithic Vertical Slices to a distributed, resilient, and fully observable microservices ecosystem powered by Event-Driven Architecture, CQRS, and eventual consistency.

Clean boundaries, immutable integration contracts, and architectural discipline are the foundation of world-class software engineering with the **Hexagen Ecosystem**.